WEB APPLICATION
SECURITY ASSESSMENT REPORT

Target Website	http://demo.testfire.net/
Tester Name	Sujal Wasnik
Assessment Period:	6–9 August 2026

1. Scope & Methodology

This assessment was carried out as a manual black-boxSecurityTesting.This is abanking application at http://demo.testfire.net/. Testing covered authentication, session management, input handling, access control, information disclosure, and HTTP security configuration.

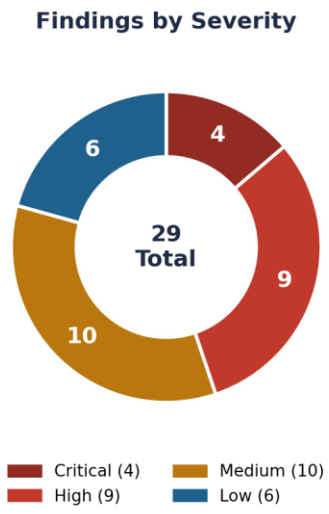
Tools used:web browser (DevTools), Burp Suite Community Edition (Proxy/Repeater), manual payload testing.

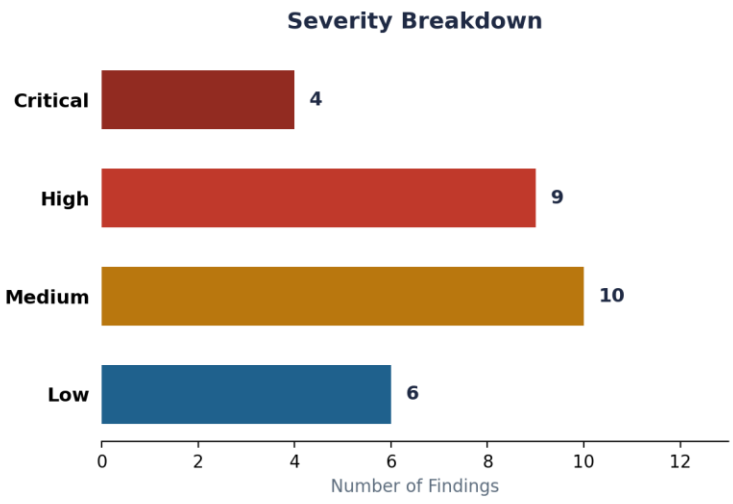
How to read this report:each finding below lists the vulnerability name, its severity, the affected URL, a plain-language description of the issue and its impact, exact steps to reproduce it, a screenshot as proof, and a recommended fix.

2. Executive Summary

A total of 29 security findings were identified during this assessment, ranging from Critical authentication-bypass and broken-access-control issues to Low-severity hardening gaps. The most serious issues allow an attacker to log in as an administrator without valid credentials (SQL Injection), view or modify other customers' bank accounts (IDOR / Broken Access Control), and reach admin-only functionality as a normal user. These should be prioritized for remediation. Findings in this report are ordered by severity, Critical first.

Severity	Count	What it means
Critical	4	Immediate, severe impact — full account takeover, unauthorized money movement, or admin access. Fix first.
High	9	Serious impact — data exposure, session/credential compromise, or code execution risk.
Medium	10	Meaningful risk that aids attackers or weakensdefences; fix in the next release cycle.
Low	6	Hardening gaps; lower immediate risk but should still be fixed.





3. Findings Summary Table

#	Vulnerabilities	Severity	Path
1	Admin Panel Accessible Without Authorization	Critical	.../admin/admin.jsp
2	SQL Injection — Login Authentication Bypass	Critical	.../login.jsp
3	IDOR — Unauthorized Access to Other Users' Bank Accounts	Critical	.../bank/showAccount?listAccounts=<account_number>
4	Broken Access Control — Fund Transfer Request Tampering	Critical	.../bank/transfer.jsp
5	SQL Injection Risk — Backend Field Mapping (doLogin)	High	.../doLogin
6	Session Fixation	High	.../login.jsp
7	Reflected XSS in Search Box	High	.../search.jsp
8	Reflected XSS via URL Parameter	High	.../search.jsp?query=<script>alert(1)</script>
9	Stored/Reflected XSS in Feedback Form — Name Field	High	.../feedback.jsp
10	XSS via Image Tag (onerror handler)	High	.../search.jsp
11	Sensitive Account Data Stored Unencrypted in Client-Side Cookie	High	.../
12	Login Credentials Sent Over Unencrypted HTTP	High	.../login.jsp
13	Directory Traversal — Access to web.xml Configuration File	High	.../index.jsp?content=../WEB-INF/web.xml
14	Swagger / API Documentation Exposed	Medium	.../swagger/index.html
15	Swagger API Schema (properties.json) Exposed	Medium	.../swagger/properties.json
16	Open Redirect via disclaimer.htm	Medium	.../disclaimer.htm?url=...
17	No Account Lockout Policy	Medium	.../login.jsp
18	Missing Cookie Security Flags	Medium	.../
19	Verbose Error Messages / Stack Trace Disclosure	Medium	.../bank/showAccount?listAccounts=ERROR
20	Reference to Legacy High-Risk Endpoint (cgi.exe)	Medium	.../cgi.exe
21	Missing Security Response Headers	Medium	.../
22	Missing CSRF Protection on Forms	Medium	.../
23	HTML Injection via content Parameter	Medium	.../index.jsp?content= HTML
24	Server Status Check Service Exposed	Low	.../util/serverStatusCheckService.jsp
25	Information Disclosure in HTML Source Comments	Low	.../login.jsp

#	Vulnerabilities	Severity	Path
26	HTTP Strict Transport Security (HSTS) Header Missing	Low	.../
27	Referrer-Policy Header Missing	Low	.../
28	Server Version Information Disclosure	Low	.../
29	Information Disclosure — Internal Executives & Management Details	Low	.../index.jsp?content=inside_executives.htm

4. Detailed Findings

Finding #1: Admin Panel Accessible Without Authorization

Severity	**Critical**
Affected URL	http://demo.testfire.net/admin/admin.jsp

Description & Impact

A regular, non-admin logged-in user can browse directly to the admin panel and reach functions meant for administrators only: adding accounts to any user, changing any user's password, and creating new users.

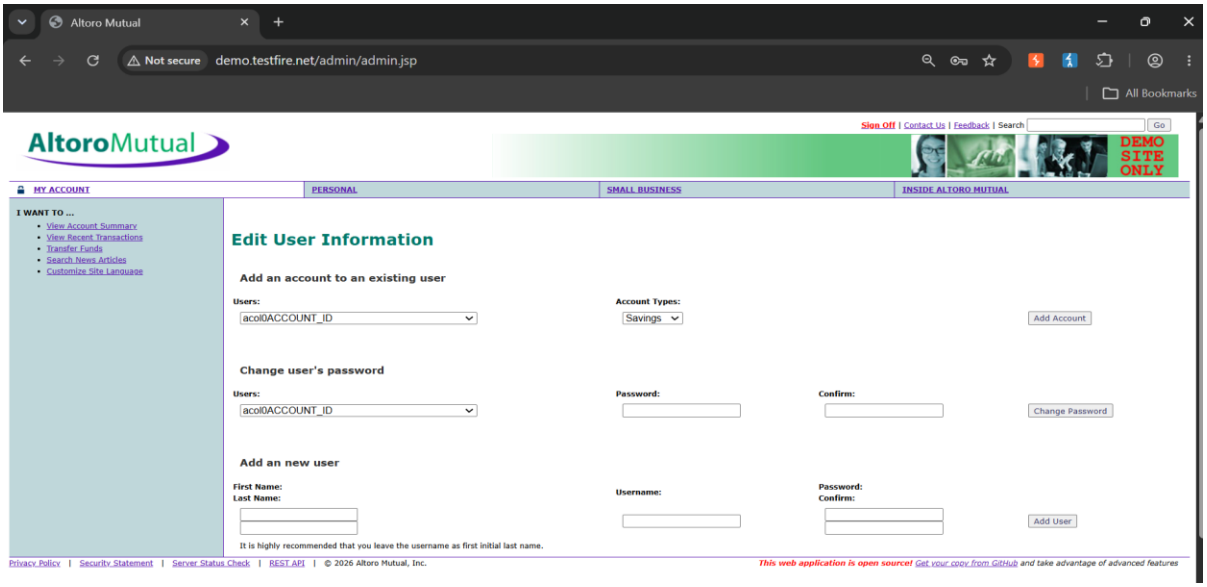
Steps to Reproduce

1.Log in with an ordinary (non-admin) account.

2.In the browser address bar, go directly to /admin/admin.jsp

3.Observe that the "Edit User Information" admin page loads, with working controls to add accounts, change any user's password, and add new users.

Screenshot / Proof of Concept (PoC)



Remediation

Enforce server-side role/permission checks on every admin endpoint. A user's role must be verified on each request, not assumed from menu visibility.

Finding #2: SQL Injection — Login Authentication Bypass *(TestLog Ref: Bug 6)*

Severity	Critical
Affected URL	http://demo.testfire.net/login.jsp (POST to /doLogin)

Description & Impact

The login form's username field is not sanitized before being used in the backend database query. Submitting a classic SQL injection payload as the username bypasses the password check entirely and logs the attacker in as an administrator — no valid credentials needed.

Steps to Reproduce

1.Go to http://demo.testfire.net/login.jsp

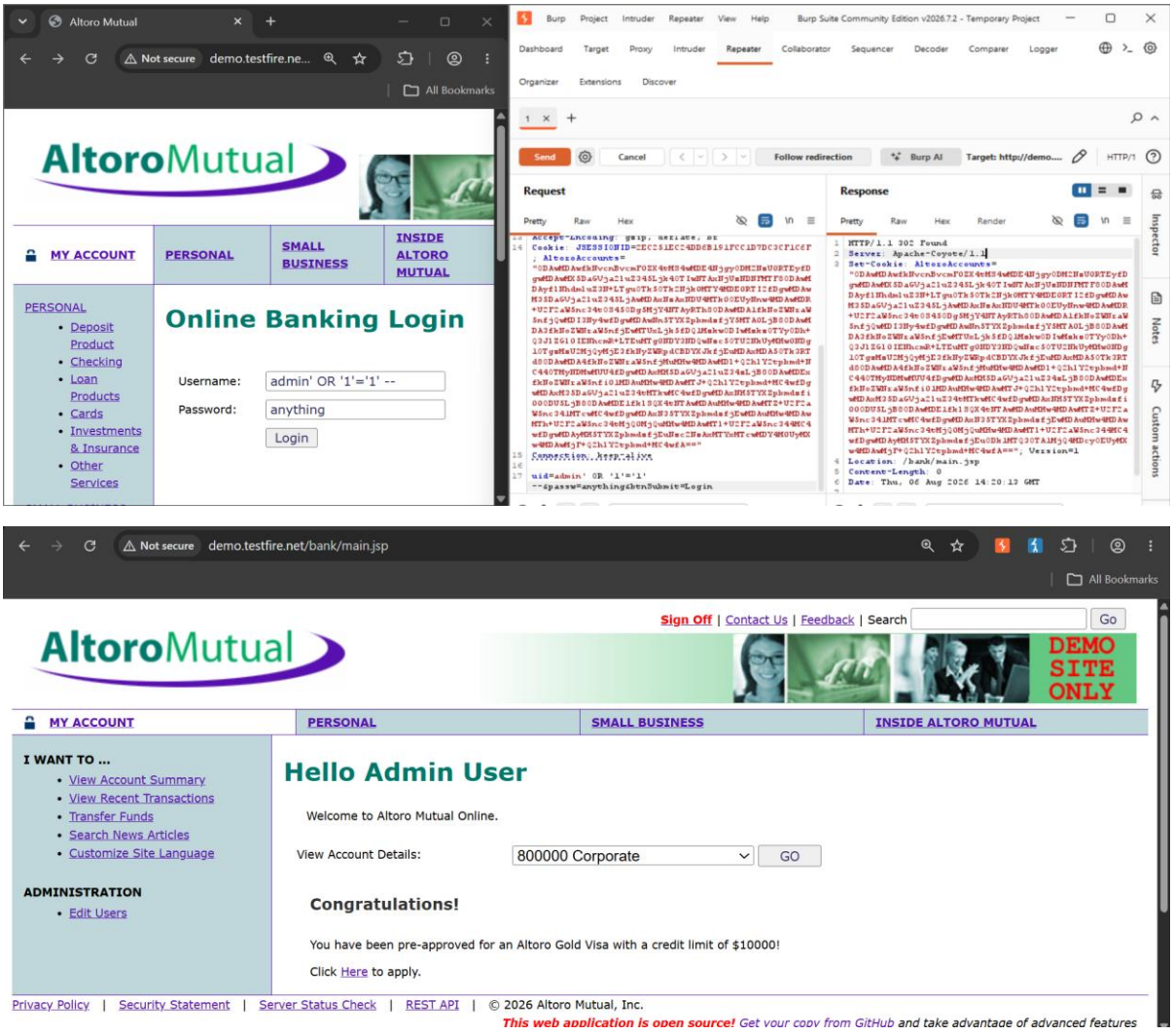
2.In the Username field, enter: admin' OR '1'='1' --

3.In the Password field, enter anything (e.g. "anything").

4.Click Login.

5.Result: the server responds 302 Found and redirects to /bank/main.jsp, logging the attacker in as "Admin User" — confirmed by the intercepted request/response in Burp Suite and the resulting "Hello Admin User" account page.

Screenshot / Proof of Concept (PoC)



Remediation

Use parameterized queries / prepared statements (never concatenate user input into SQL). Apply strict input validation and least-privilege database accounts, and add a WAF rule as defense-in-depth.

Finding #3: IDOR — Unauthorized Access to Other Users' Bank Accounts

Severity	**Critical**
Affected URL	http://demo.testfire.net/bank/showAccount?listAccounts=800002

Description & Impact

After logging in, the `listAccounts` parameter that selects which account to view is not checked against the logged-in user's own accounts. Simply changing the account number in the URL discloses another customer's account history and balance.

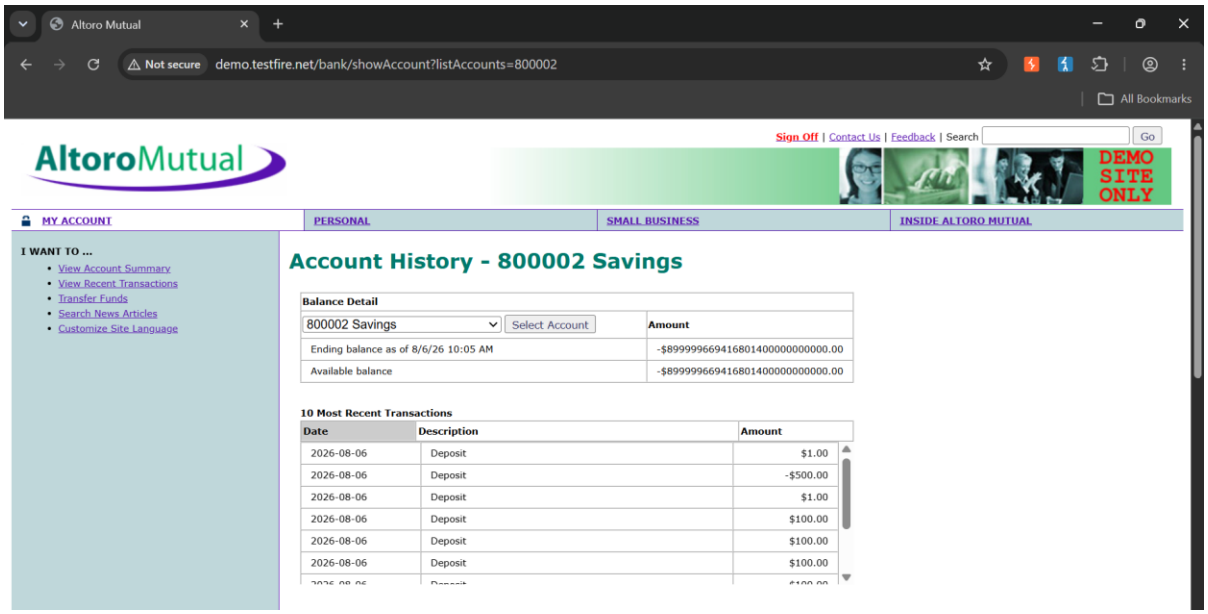
Steps to Reproduce

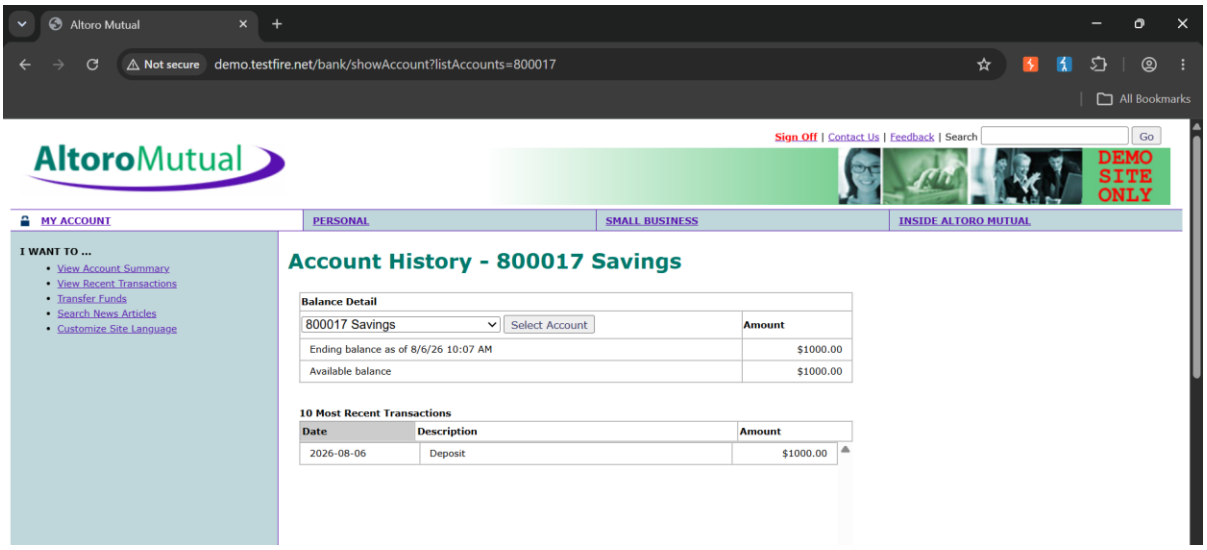
1.Log in and go to: /bank/showAccount?listAccounts=800002 — note this shows the logged-in user's own "800002 Savings" account.

2.Change the number in the URL to a different account, e.g.: /bank/showAccount?listAccounts=800017

3.Result: the page loads "Account History - 800017 Savings" showing that account's balance and transactions — an account that does not belong to the logged-in user, with no error or access check.

Screenshot / Proof of Concept (PoC)





Remediation

Verify server-side that the requested account number belongs to the authenticated user before returning any account data (object-level authorization check on every request).

Finding #4: Broken Access Control — Fund Transfer Request Tampering

Severity	**Critical**
Affected URL	http://demo.testfire.net/bank/transfer.jsp (POST to /bank/doTransfer)

Description & Impact

By intercepting the "Transfer Funds" request with a proxy tool, the source/destination account fields (`fromAccount`, `toAccount`) and amount can be freely modified and replayed. The server accepts the tampered request and completes the transfer without verifying that the accounts belong to the authenticated user, confirming a broken server-side access control on a sensitive money-movement function.

Steps to Reproduce

1.Log in and open Transfer Funds with a proxy (e.g. Burp Suite) intercepting traffic.

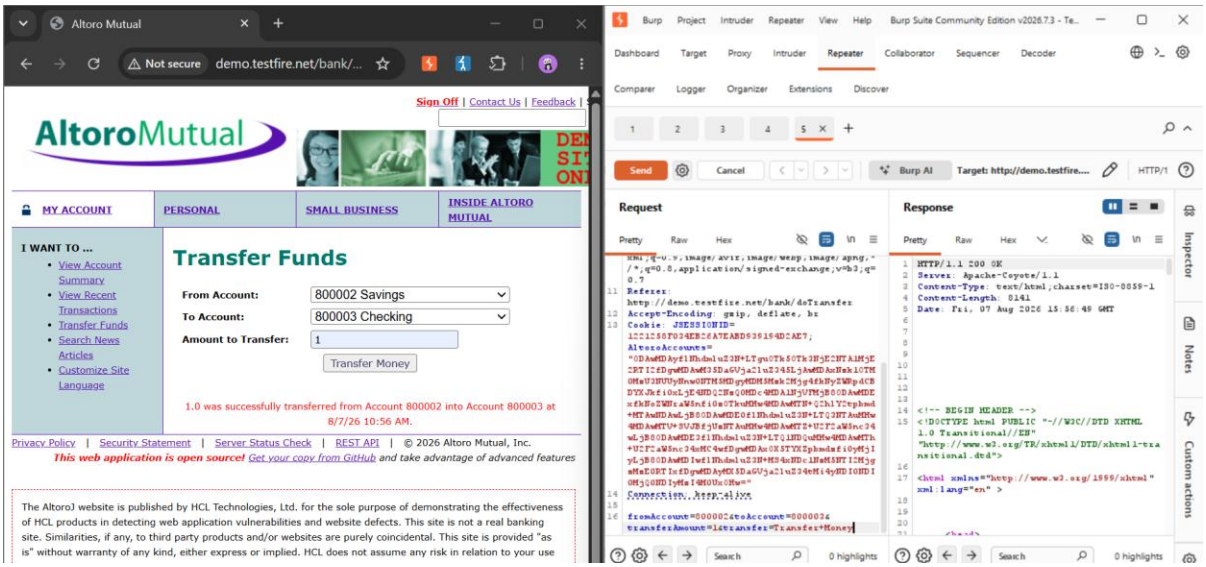
2.Capture the transfer POST request to /bank/doTransfer.

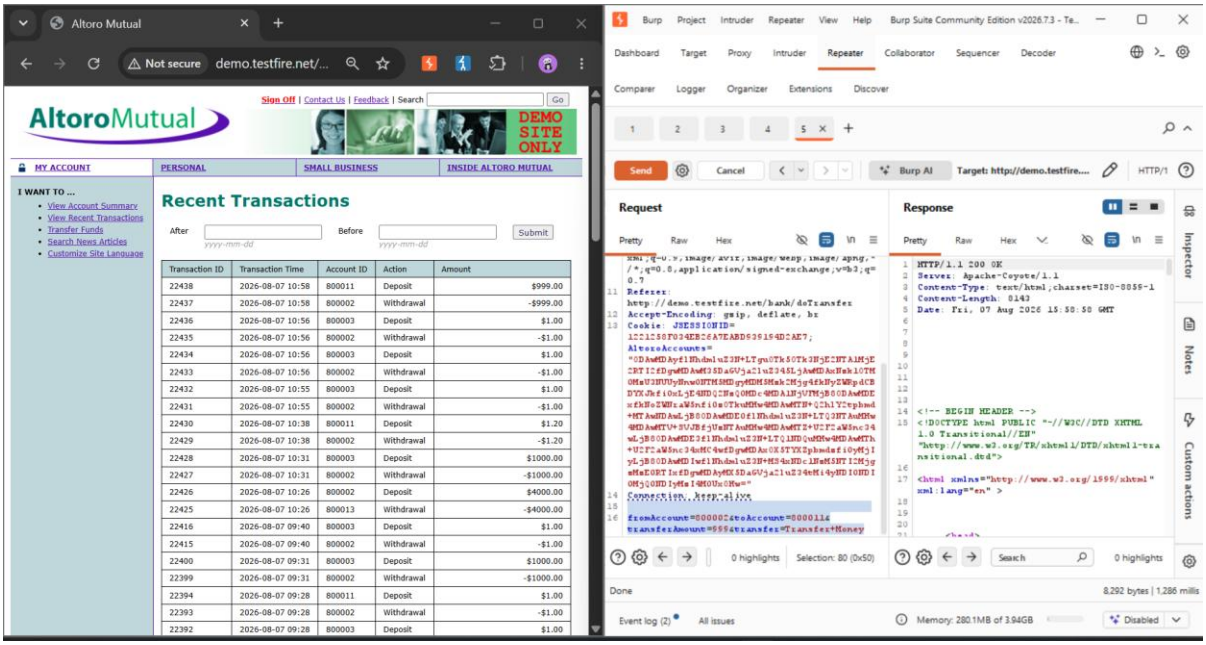
3.Modify the fromAccount / toAccount / transferAmount parameters

(e.g. fromAccount=800002&toAccount=800011&transferAmount=999) and forward the request.

4.Result: the server responds 200 OK and the "View Recent Transactions" page shows the corresponding deposit/withdrawal entries — the tampered transfer succeeded.

Screenshot / Proof of Concept (PoC)





Remediation

Enforce server-side authorization checks that the authenticated user actually owns `fromAccount` before processing any transfer, and re-validate all transaction parameters server-side regardless of what the client submits.

Finding #5: SQL Injection Risk — Backend Field Mapping (doLogin)

Severity	High
Affected URL	http://demo.testfire.net/doLogin

Description & Impact

Inspecting the raw login request shows the form posts simplified field names (`uid`, `passw`) that map directly onto a backend query pattern such as `SELECT * FROM users WHERE username='uid' AND password='passw'`. This is the same root cause as Bug 6 — confirmation that every input reaching /doLogin needs to be treated as a SQL injection vector, not just the username field.

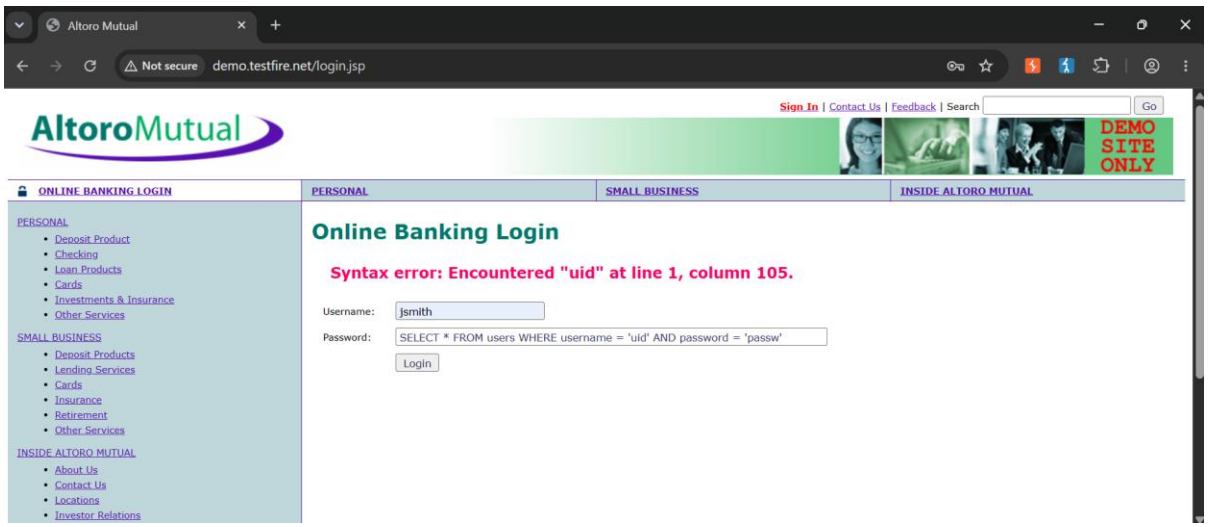
Steps to Reproduce

1.Intercept a login request to /doLogin using a proxy tool (e.g. Burp Suite).

2.Observe the POST body parameters `uid` and `passw`.

3.Confirm (as in Bug 6) that injecting SQL metacharacters into either field alters query logic.

Screenshot / Proof of Concept (PoC)



Remediation

Same as Bug 6: parameterized queries on all fields submitted to /doLogin, plus server-side input validation.

Finding #6: Session Fixation

Severity	High
Affected URL	http://demo.testfire.net/login.jsp

Description & Impact

The session identifier (JSESSIONID) issued to a visitor before login is not regenerated after a successful login. An attacker who can plant a known session ID in a victim's browser before they log in can hijack that same session afterward.

Steps to Reproduce

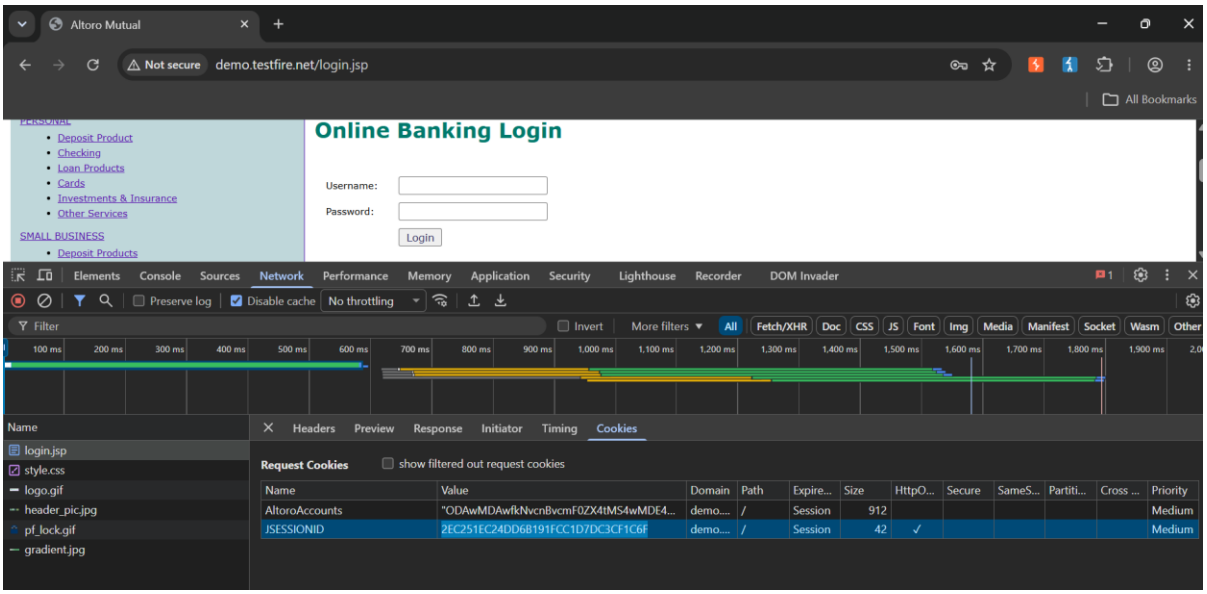
1.Before logging in, open DevTools → Application → Cookies and note the JSESSIONID value.

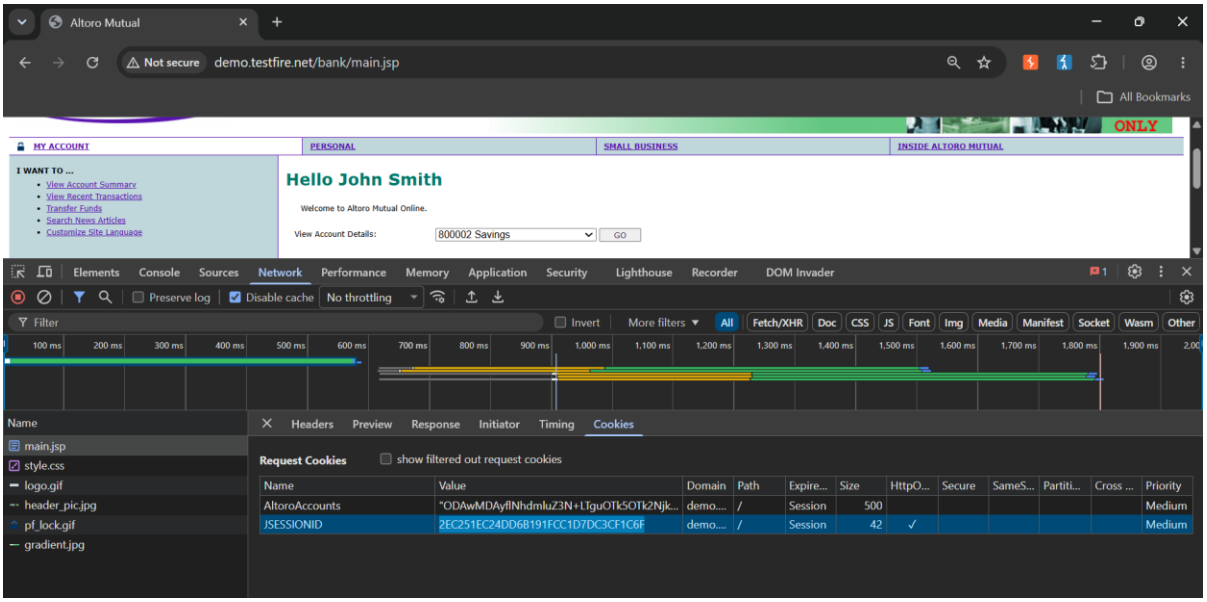
2.Log in with valid credentials.

3.Check the JSESSIONID cookie again after login.

4.Result: the JSESSIONID value(2EC251EC24DD6B191FCC1D7DC3CF1C6F)is identical before and after login — confirmed via the browser DevTools cookie panel in both screenshots.

Screenshot / Proof of Concept (PoC)





Remediation

Regenerate the session ID immediately after a successful authentication event (standard behavior in most modern frameworks — do not reuse the pre-login session).

Finding #7: Reflected XSS in Search Box

Severity	High
Affected URL	http://demo.testfire.net/search.jsp

Description & Impact

Input submitted through the site's search box is reflected back into the page without encoding, allowing arbitrary JavaScript to execute in the victim's browser.

Steps to Reproduce

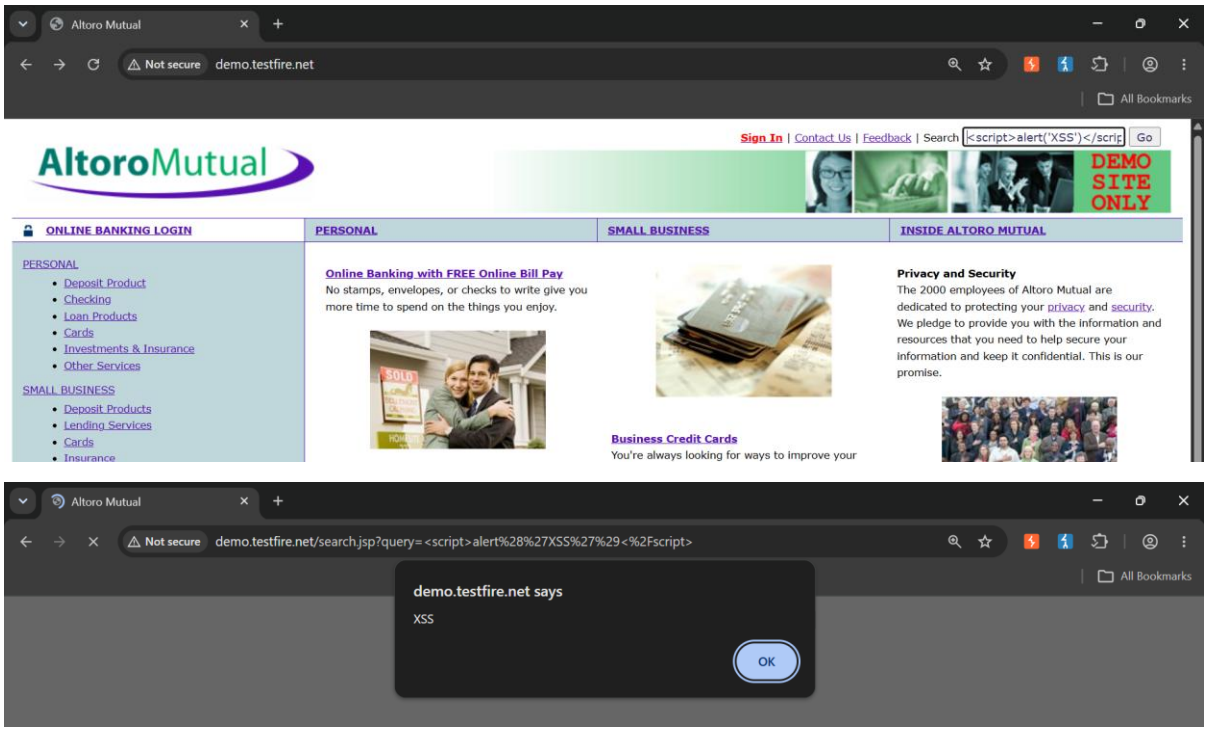
1.Go to http://demo.testfire.net/search.jsp

2.In the search box, enter: <script>alert('XSS')</script>

3.Click Search/Go.

4.Result: the injected script executes / is reflected in the page output.

Screenshot / Proof of Concept (PoC)



Remediation

Encode all user-supplied output for the context it's rendered in (HTML-entity encode for HTML body), and apply a Content-Security-Policy as defense-in-depth.

Finding #8: Reflected XSS via URL Parameter

Severity	High
Affected URL	http://demo.testfire.net/search.jsp?query=<script>alert(1)</script>

Description & Impact

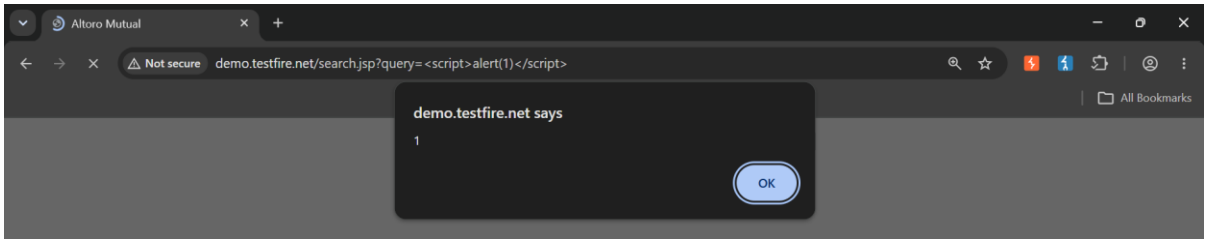
The `query` URL parameter is reflected into the page unencoded, so a crafted link alone (no form submission needed) triggers script execution — this is the most "shareable" form of XSS since the payload lives entirely in the URL.

Steps to Reproduce

1.Go to: http://demo.testfire.net/search.jsp?query=<script>alert(1)</script>

2.Result: a JavaScript alert box fires immediately on page load, showing "1" — confirmed in the screenshot.

Screenshot / Proof of Concept (PoC)



Remediation

Same as Bug 11 — context-appropriate output encoding on the `query` parameter, plus CSP.

Finding #9: Stored/Reflected XSS in Feedback Form — Name Field

Severity	High
Affected URL	http://demo.testfire.net/feedback.jsp

Description & Impact

The "Name" field on the feedback form does not sanitize input before it is rendered back to the user, allowing script injection.

Steps to Reproduce

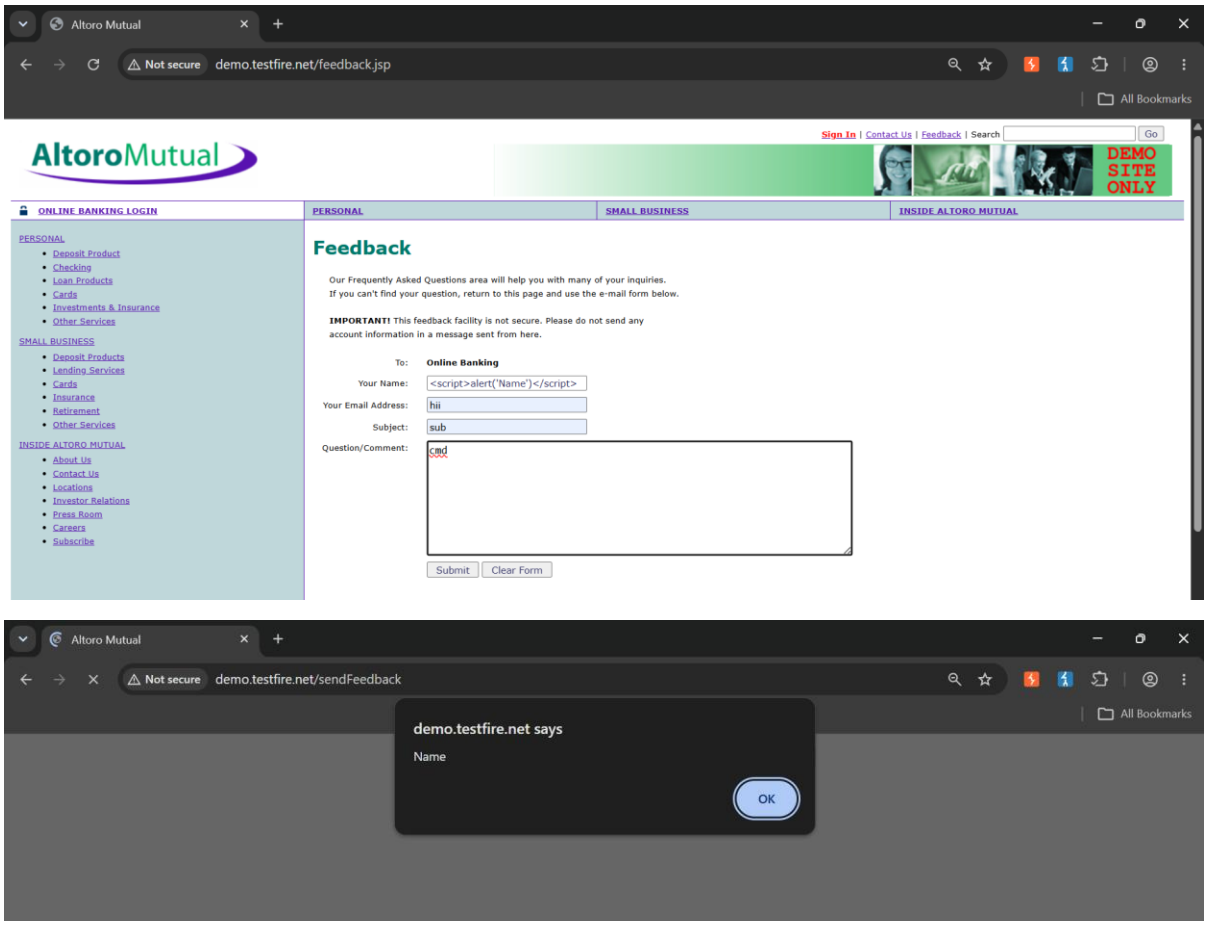
1.Go to http://demo.testfire.net/feedback.jsp

2.In the "Name" field, enter: <script>alert('Name')</script>

3.Fill in the remaining required fields and submit.

4.Result: the injected script executes / is reflected back in the confirmation output.

Screenshot / Proof of Concept (PoC)



Remediation

Encode all form field output before rendering, and validate/sanitize input server-side regardless of client-side checks.

Finding #10: XSS via Image Tag (onerror handler)

Severity	**High**
Affected URL	http://demo.testfire.net/search.jsp (and feedback.jsp)

Description & Impact

The same unsanitized-output issue also allows the classic `<img src=x onerror=...>` bypass, which is important because it shows basic `<script>` tag filtering (if any existed) would not be sufficient — the app has no real output encoding at all.

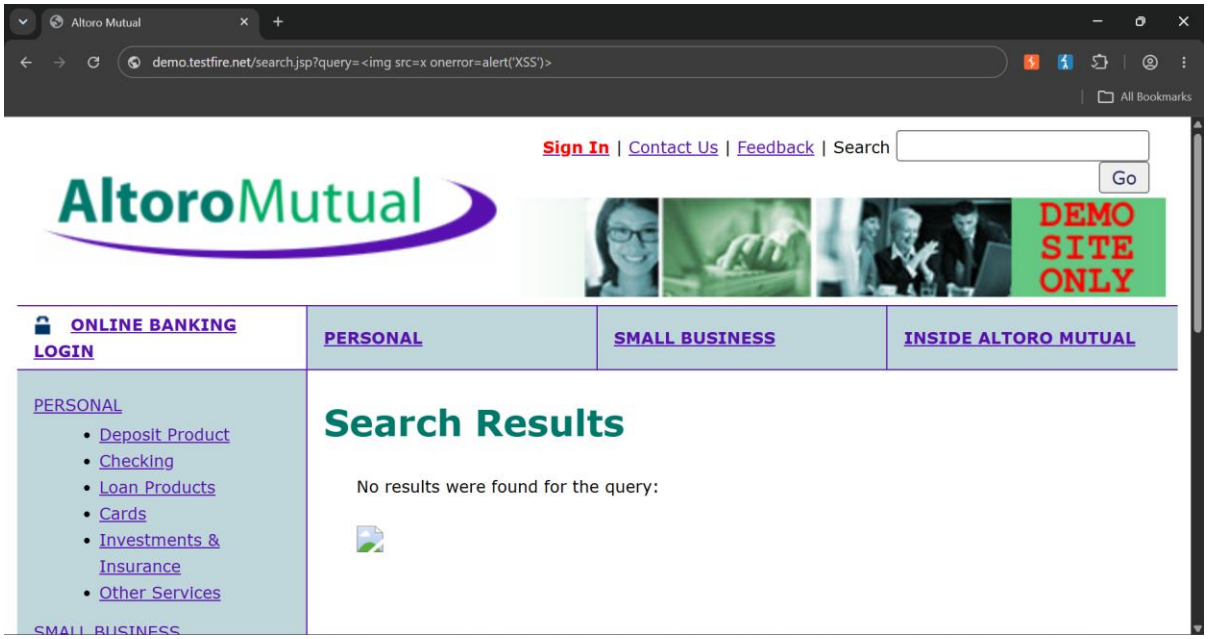
Steps to Reproduce

1.In the search box or feedback form, enter: <img src=x onerror=alert('XSS')>

2.Submit.

3.Result: the onerror handler fires and the alertexecute, confirming no encoding/filtering is in place.

Screenshot / Proof of Concept (PoC)



Remediation

Same as Bugs 11–13: proper output encoding is the fix, not tag-name blacklisting.

Finding #11: Sensitive Account Data Stored Unencrypted in Client-Side Cookie

Severity	**High**
Affected URL	http://demo.testfire.net/ (AltoroAccounts cookie)

Description & Impact

The `AltoroAccounts` cookie contains a Base64-encoded (not encrypted) string that, once decoded, reveals raw account structure: account numbers, account types, and balances (e.g. `800002~Savings~-8.99...|800003~Checking~...`). Because this data lives client-side and is only encoded (trivially reversible) rather than encrypted, a user can view or tamper with their own account references, increasing the risk of IDOR and parameter-tampering attacks.

Steps to Reproduce

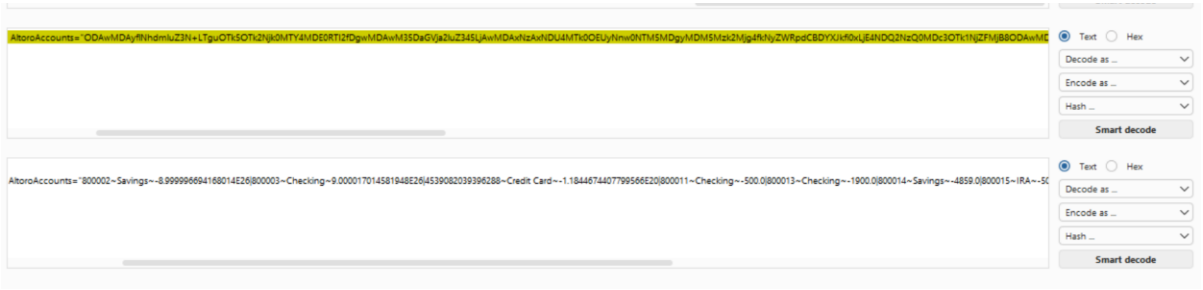
1.Log in and open DevTools → Application → Cookies.

2.Copy the value of the AltoroAccounts cookie.

3.Base64-decode the value (e.g. using DevTools console or an online decoder).

4.Result: the decoded value reveals plaintext account numbers, account types, and balances.

Screenshot / Proof of Concept (PoC)



Remediation

Do not store sensitive account/balance data in client-side cookies at all. Keep this state server-side, keyed to the authenticated session, and re-validate ownership on every request.

Finding #12: Login Credentials Sent Over Unencrypted HTTP

Severity	High
Affected URL	http://demo.testfire.net/login.jsp

Description & Impact

The entire site, including the login form, is served over plain HTTP rather than HTTPS. Login credentials are transmitted in clear text and can be captured by anyone able to observe network traffic (e.g. on shared/public Wi-Fi).

Steps to Reproduce

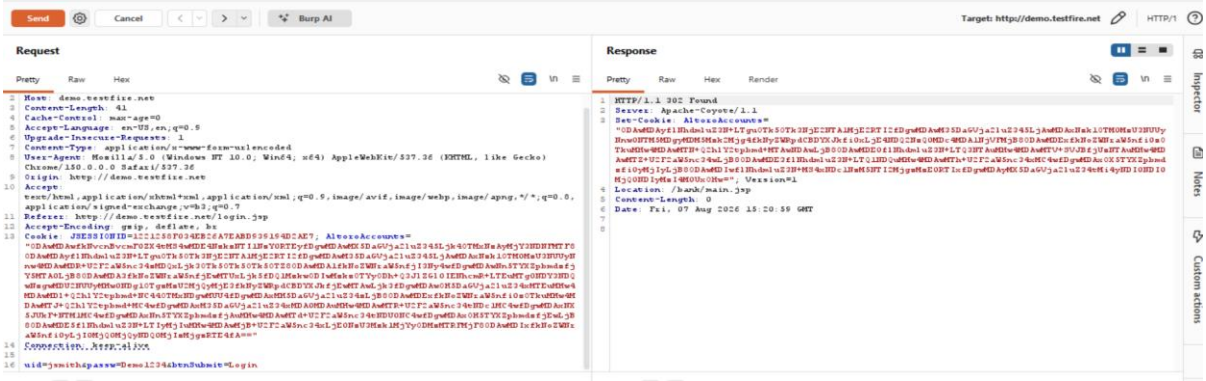
1.Open DevTools → Network (or a proxy tool) before logging in.

2.Submit the login form with any credentials.

3.Inspect the captured request.

4.Result: the connection is HTTP (not HTTPS), so the username and password are visible in plain text in the captured traffic.

Screenshot / Proof of Concept (PoC)



Remediation

Serve the entire application over HTTPS/TLS only, redirect all HTTP traffic to HTTPS, and enable HSTS (see Bug 23).

Finding #13: Directory Traversal — Access to web.xml Configuration File

Severity	High
Affected URL	http://demo.testfire.net/index.jsp?content=../WEB-INF/web.xml

Description & Impact

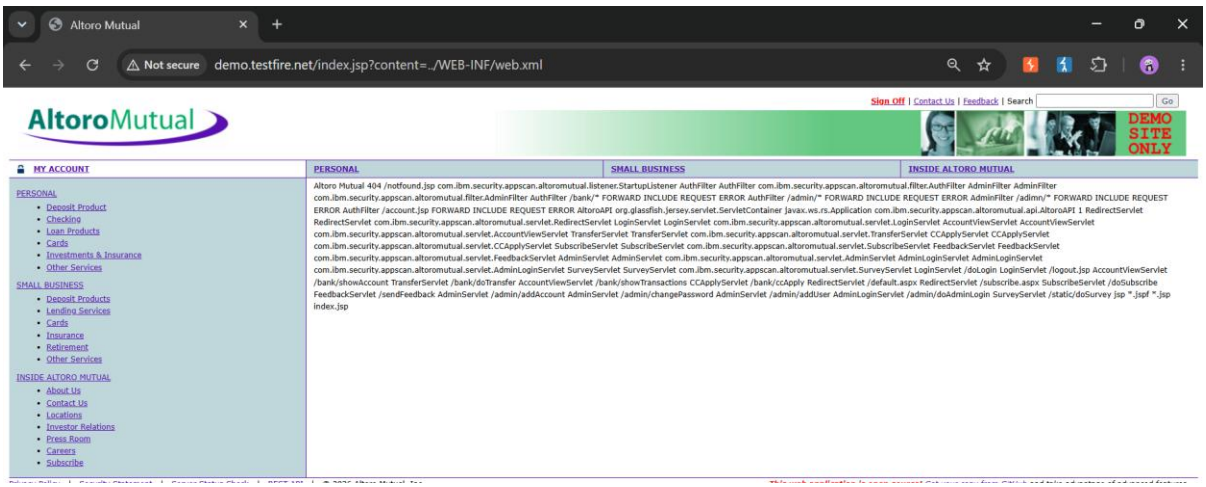
The same unsanitized `content` parameter seen in Bug 26 also allows directory-traversal sequences, letting an attacker read the application's `web.xml` configuration file directly from the server. This discloses every internal servlet mapping, class name, and initialization parameter in the application.

Steps to Reproduce

1.Go to: http://demo.testfire.net/index.jsp?content=../WEB-INF/web.xml

2.Result: the server returns the full contents of web.xml, listing every servlet, its mapped URL, and internal Java class names (e.g. AdminServlet → /admin/addUser, TransferServlet → /bank/doTransfer, etc.).

Screenshot / Proof of Concept (PoC)



Remediation

Sanitize the `content` parameter to reject path-traversal sequences (`../`), and store WEB-INF/protected resources outside the web root's reachable path resolution entirely — never resolve arbitrary user input into a filesystem path.

Finding #14: Swagger / API Documentation Exposed

Severity	Medium
Affected URL	http://demo.testfire.net/swagger/index.html

Description & Impact

The Swagger UI for the site's REST API ("AltoroJ REST API") is publicly reachable without any login. It lists every API endpoint (login, account, transactions, etc.), giving an attacker a ready-made map of the application's attack surface.

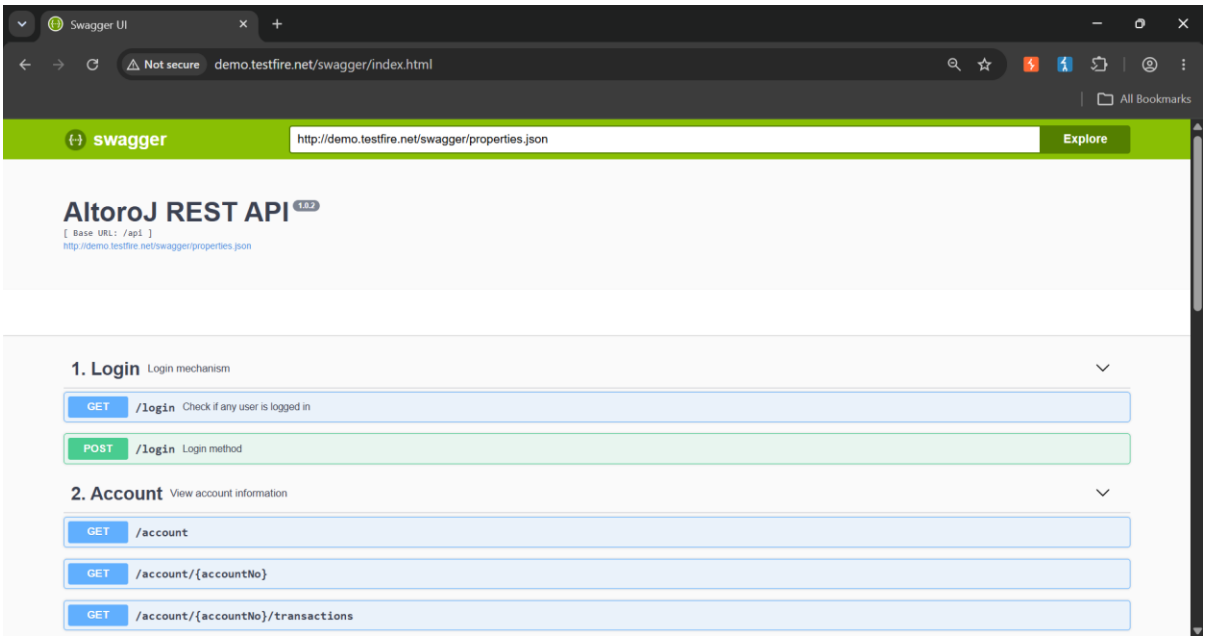
Steps to Reproduce

1.Open a browser (no login required).

2.Go to http://demo.testfire.net/swagger/index.html

3.Observe the full list of API endpoints and methods (GET/POST) displayed on the page.

Screenshot / Proof of Concept (PoC)



Remediation

Remove Swagger UI from the production/public-facing environment, or restrict access behind authentication and an allow-list of internal IPs.

Finding #15: Swagger API Schema (properties.json) Exposed

Severity	**Medium**
Affected URL	http://demo.testfire.net/swagger/properties.json

Description & Impact

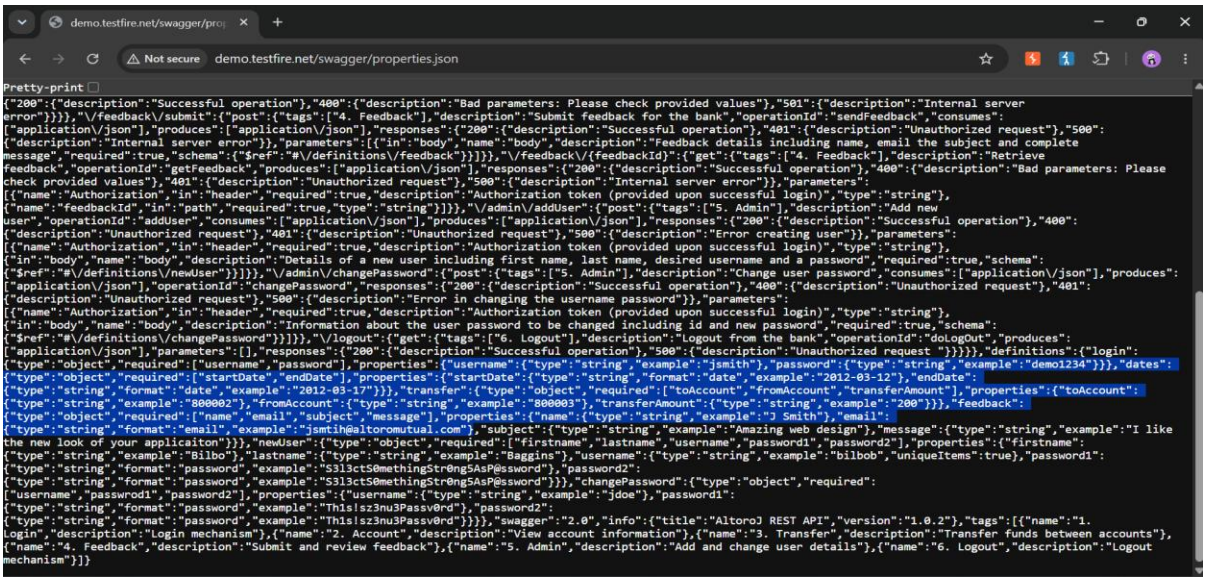
The underlying Swagger schema file is directly downloadable without authentication, exposing the raw API definition including parameter names and endpoint structure.

Steps to Reproduce

1.Go to http://demo.testfire.net/swagger/properties.json

2.Observe that the raw JSON API schema is returned directly in the browser.

Screenshot / Proof of Concept (PoC)



Remediation

Same as Bug 1 — restrict or remove public access to Swagger/OpenAPI definition files.

Finding #16: Open Redirect via disclaimer.htm

Severity	**Medium**
Affected URL	http://demo.testfire.net/disclaimer.htm?url=...

Description & Impact

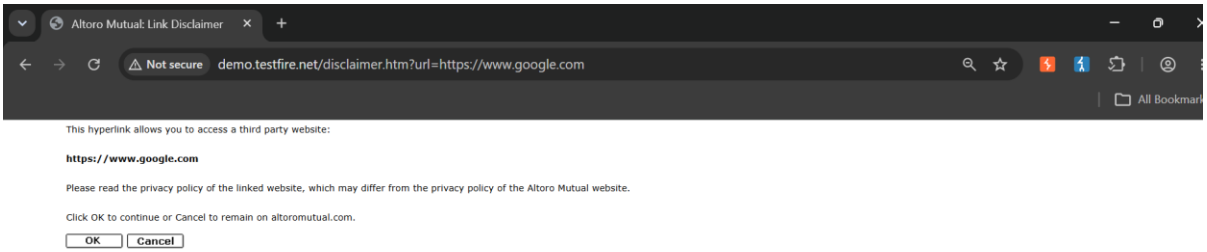
The disclaimer page takes a `url` parameter and redirects the visitor to it without validating that the destination is a trusted, internal address. This can be abused to craft links that appear to start on the legitimate bank domain but send victims to an attacker-controlled site (useful for phishing).

Steps to Reproduce

1.Go to http://demo.testfire.net/disclaimer.htm?url=https://www.google.com/

2.Observe that the site redirects/links onward to the supplied URL without validation.

Screenshot / Proof of Concept (PoC)



Remediation

Validate the `url` parameter against an allow-list of trusted internal destinations, or remove the open redirect entirely and use fixed internal links.

Finding #17: No Account Lockout Policy

Severity	Medium
Affected URL	http://demo.testfire.net/login.jsp

Description & Impact

The login endpoint does not lock or throttle an account after repeated failed login attempts, which makes online password-guessing (brute-force) attacks practical.

Steps to Reproduce

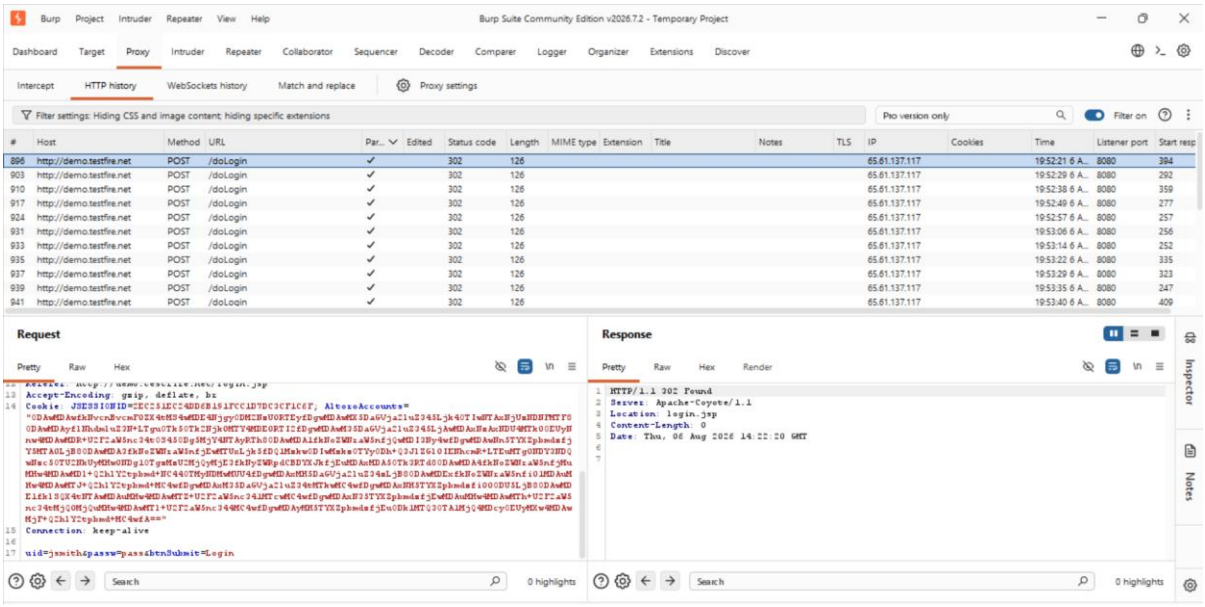
1.Go to the login page.

2.Enter a valid username with an incorrect password and submit. Repeat 10+ times in a row.

3.Try an 11th attempt with the same wrong password.

4.Result: the request log shows repeated /doLogin POSTs all returning 302 Found back to login.jsp with no lockout, delay, or CAPTCHA triggered at any point.

Screenshot / Proof of Concept (PoC)



Remediation

Implement account lockout or exponential back-off after a defined number of failed attempts, plus CAPTCHA and login attempt monitoring/alerting.

Finding #18: Missing Cookie Security Flags

Severity	Medium
Affected URL	http://demo.testfire.net/ (JSESSIONID cookie)

Description & Impact

The session cookie is missing the `Secure` and `SameSite` attributes (HttpOnly is present). Without `Secure`, the cookie can be sent over unencrypted HTTP; without `SameSite`, the cookie is more exposed to cross-site request forgery.

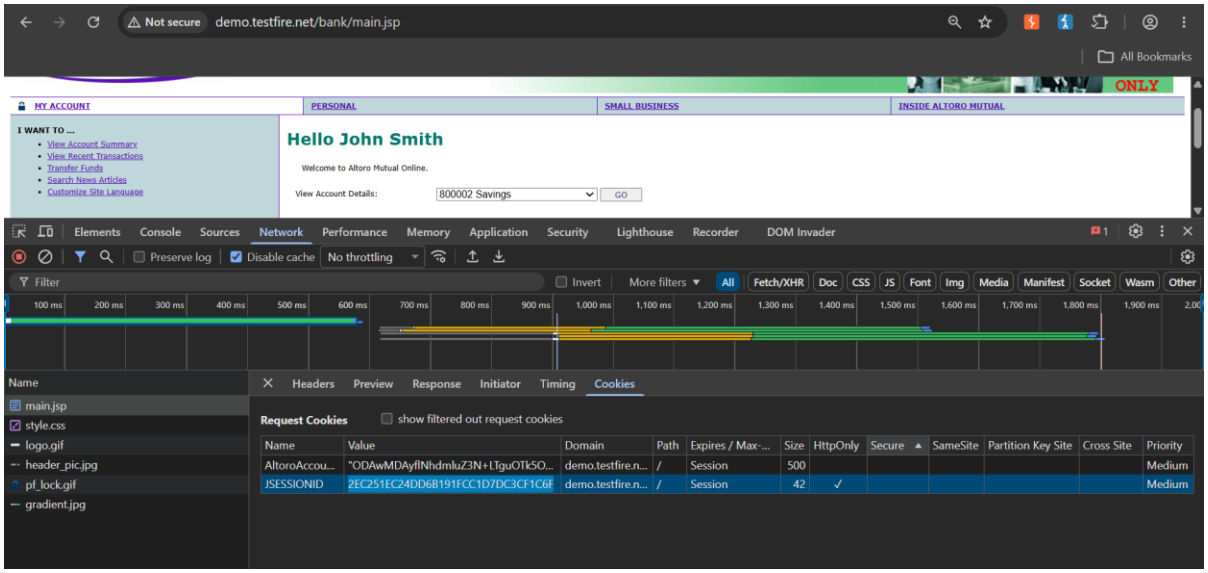
Steps to Reproduce

1.Log in, then open DevTools → Application → Cookies for demo.testfire.net.

2.Inspect the JSESSIONID cookie's flags.

3.Result: HttpOnly is set, but Secure and SameSite are not — visible directly in the cookie table.

Screenshot / Proof of Concept (PoC)



Remediation

Set Secure and SameSite (Lax or Strict) on all session cookies, and serve the entire site over HTTPS so Secure can be enforced.

Finding #19: Verbose Error Messages / Stack Trace Disclosure

Severity	**Medium**
Affected URL	http://demo.testfire.net/bank/showAccount?listAccounts=ERROR

Description & Impact

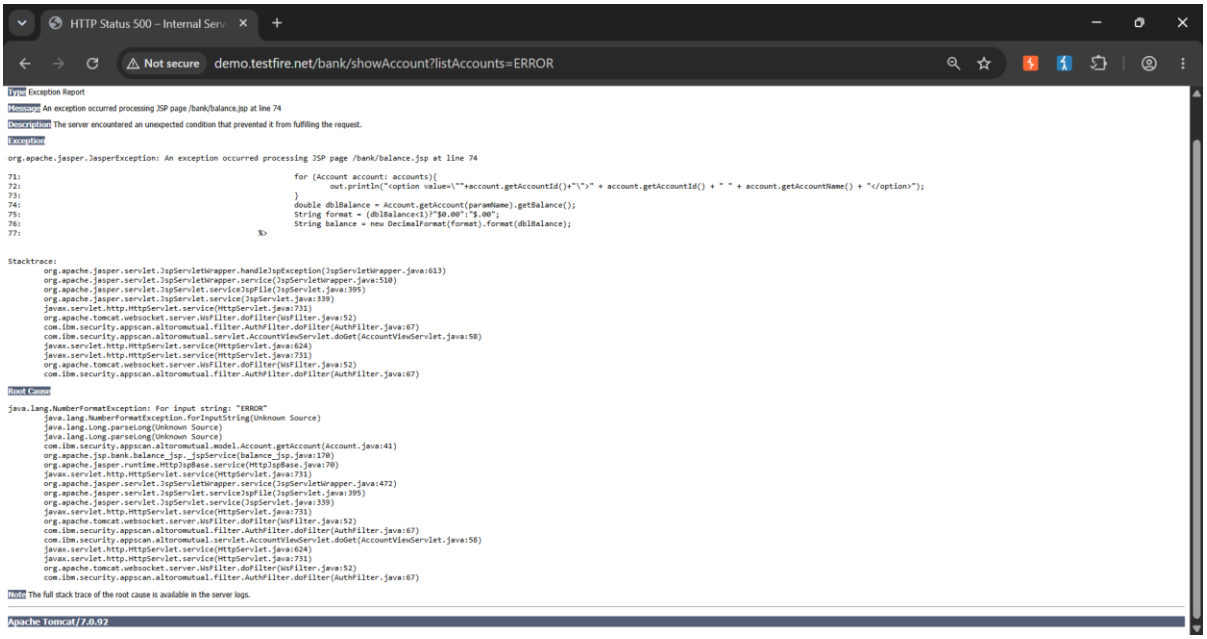
Submitting an invalid value for the account parameter triggers an unhandled server error that returns a full Java stack trace, including internal class names (e.g. `com.ibm.security.appscan.altoromutual.*`), servlet call chain, and the exact application server version (Apache Tomcat/7.0.92). This is highly valuable reconnaissance for an attacker fingerprinting the stack for known exploits.

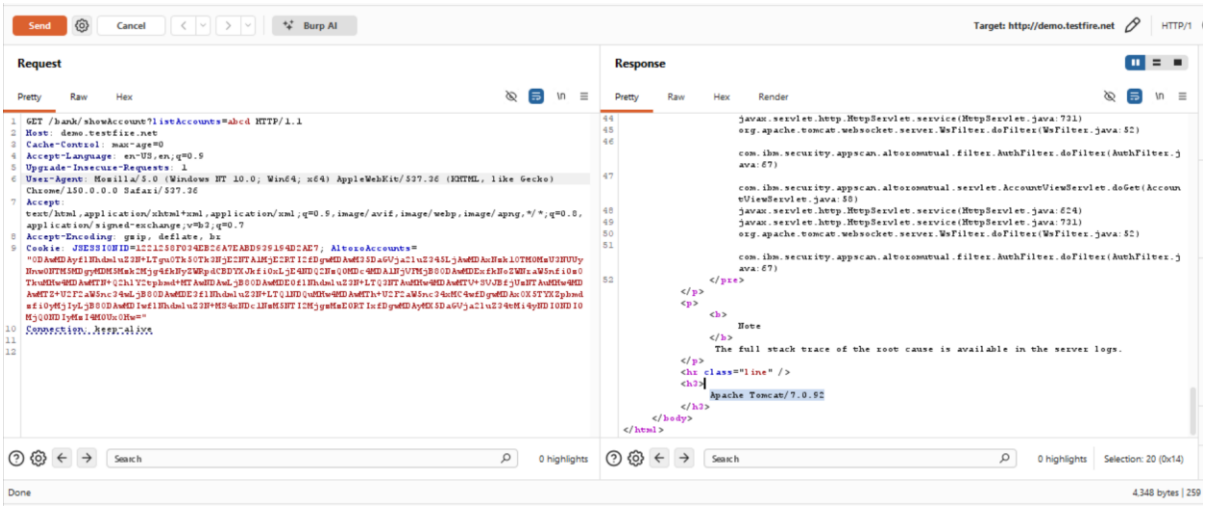
Steps to Reproduce

1.Log in and go to: /bank/showAccount?listAccounts=ERROR (or any non-numeric/invalid value).

2.Result: the server returns a detailed error page with a full stack trace, internal package/class names, and the Tomcat version number, instead of a generic error message.

Screenshot / Proof of Concept (PoC)





Remediation

Disable detailed error/stack-trace output in production (custom generic error pages only); log full details server-side, not to the client.

Finding #20: Reference to Legacy High-Risk Endpoint (cgi.exe)

Severity	Medium
Affected URL	http://demo.testfire.net/cgi.exe (linked from the "Locations" menu item)

Description & Impact

The site's navigation menu links directly to a `.exe` endpoint (`cgi.exe`). Legacy CGI executables like this often pass URL parameters straight into OS-level commands. This is flagged as an observation/high-risk area worth targeted testing for command injection (using separators such as `;`, `|`, `&&`, or backticks), rather than a confirmed exploit in this pass.

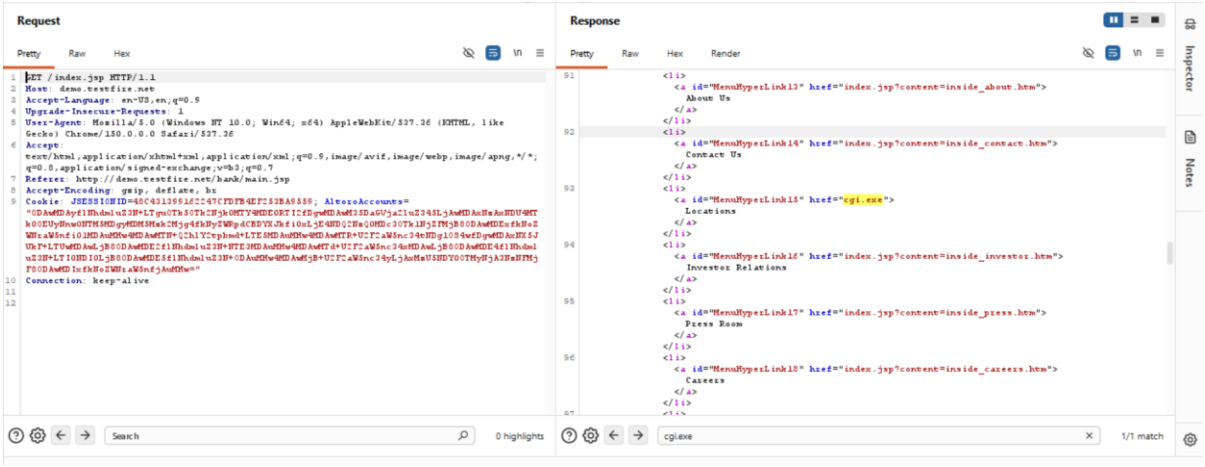
Steps to Reproduce

1.View the site's page source or sidebar navigation menu.

2.Locate the "Locations" menu item and inspect its href attribute.

3.Result: the link points directly to `cgi.exe` — an executable endpoint pattern historically associated with OS command injection vulnerabilities in legacy web servers.

Screenshot / Proof of Concept (PoC)



Remediation

Retire legacy CGI/.exe endpoints entirely if unused. If still required, ensure all parameters are strictly validated and never passed to a shell/OS command; run under least-privilege service accounts.

Finding #21: Missing Security Response Headers

Severity	**Medium**
Affected URL	http://demo.testfire.net/ (site-wide)

Description & Impact

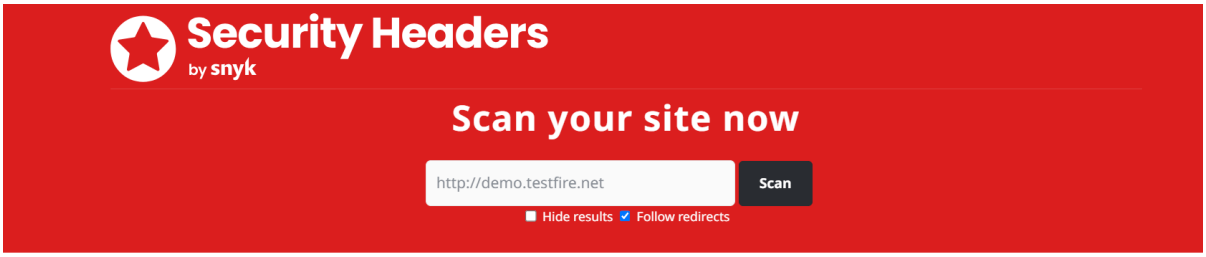
HTTP responses across the site are missing standard hardening headers: `Content-Security-Policy`, `X-Frame-Options`, and `X-Content-Type-Options`. Their absence increases exposure to XSS impact, clickjacking, and MIME-sniffing attacks.

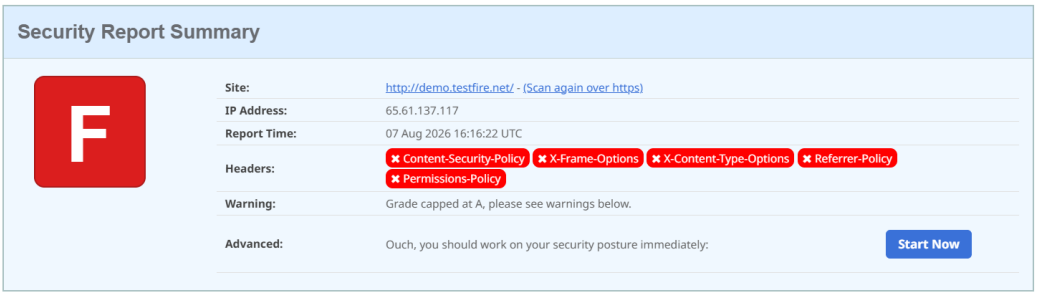
Steps to Reproduce

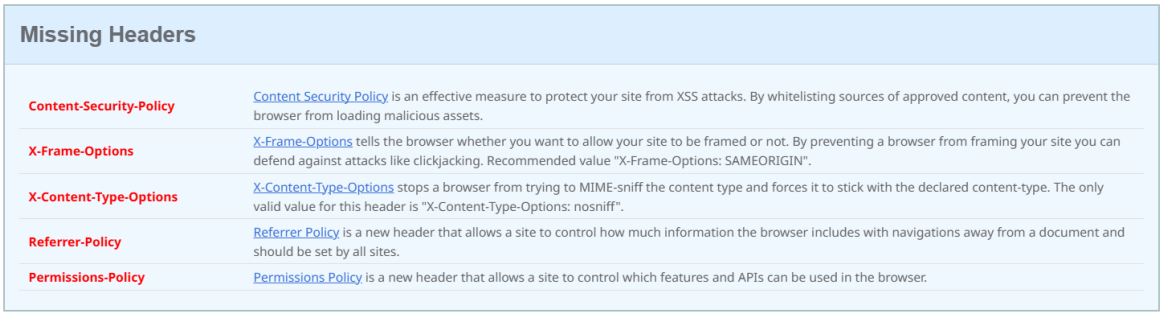
1.Load any page and inspect the HTTP response headers (DevTools → Network, or a proxy tool).

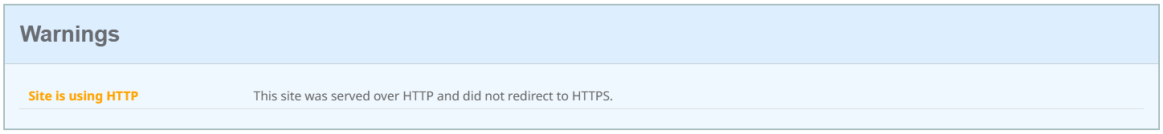
2.Result: Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options are all absent from the response.

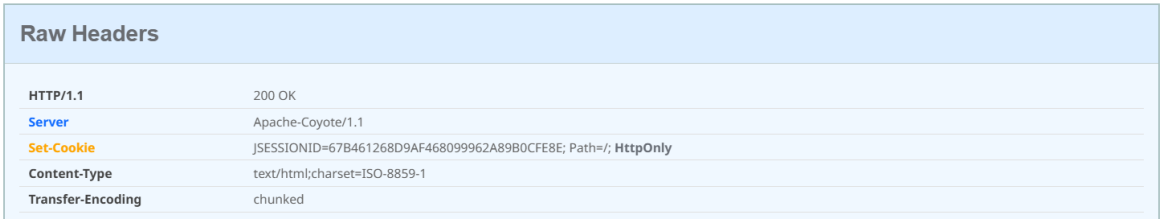
Screenshot / Proof of Concept (PoC)

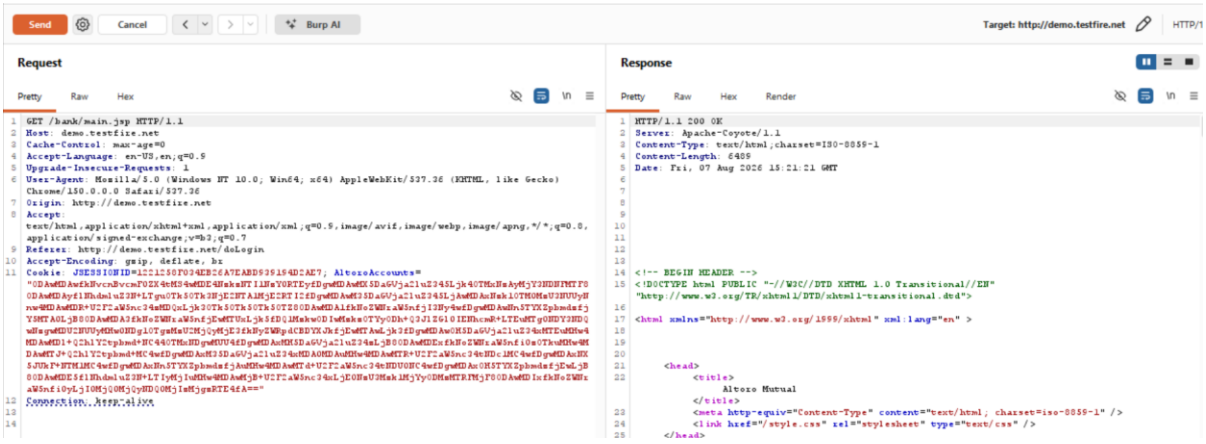












Remediation

Add Content-Security-Policy, X-Frame-Options: DENY (or SAMEORIGIN), and X-Content-Type-Options: nosniff to all HTTP responses at the server/framework level.

Finding #22: Missing CSRF Protection on Forms

Severity	Medium
Affected URL	http://demo.testfire.net/ (site-wide forms, e.g. transfer, feedback, login)

Description & Impact

Forms across the application do not include anti-CSRF tokens. Without a per-session, unpredictable token validated server-side, an attacker can craft a malicious page that causes a logged-in victim's browser to submit state-changing requests (such as fund transfers) without their consent.

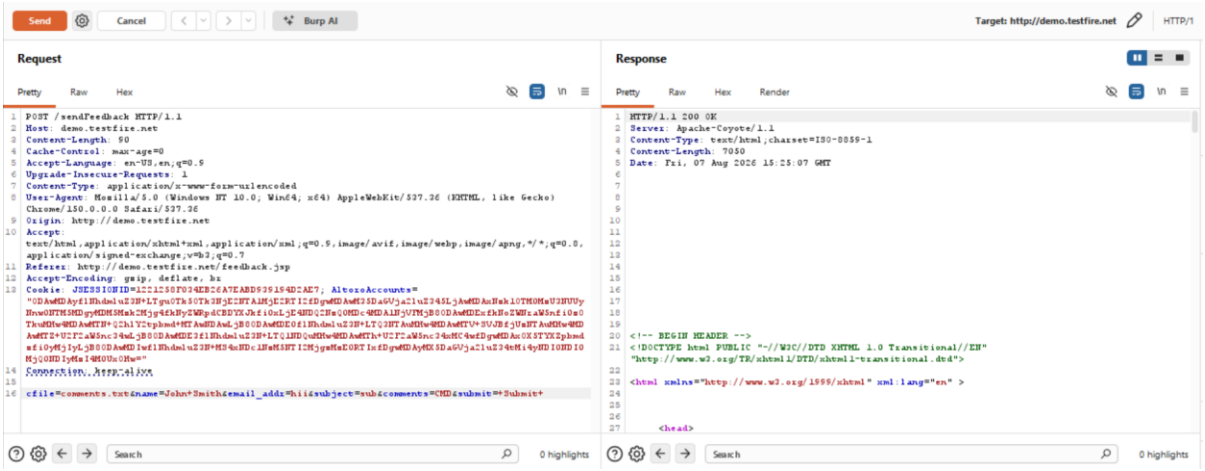
Steps to Reproduce

1.Log in and inspect the HTML of a state-changing form (e.g. Transfer Funds).

2.Search for a hidden CSRF token field in the form.

3.Result: no anti-CSRF token is present in the form submission.

Screenshot / Proof of Concept (PoC)



Remediation

Implement per-session, per-request CSRF tokens validated server-side on every state-changing request, and set SameSite on session cookies as additional defense.

Finding #23: HTML Injection via content Parameter

Severity	Medium
Affected URL	http://demo.testfire.net/index.jsp?content= HTML

Description & Impact

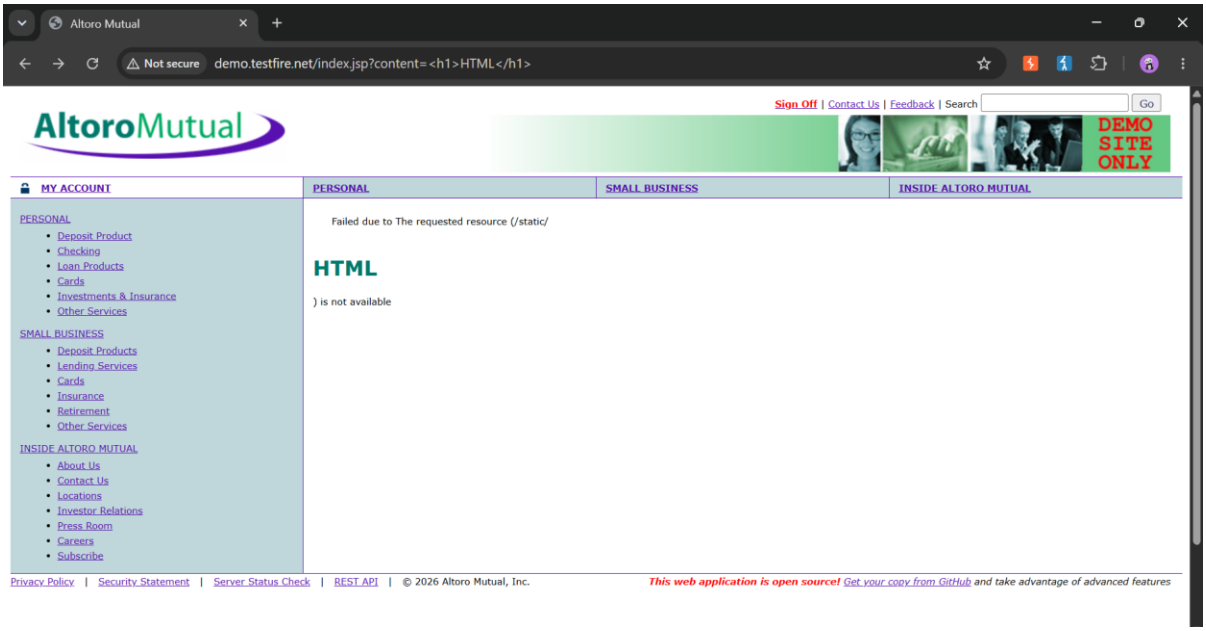
The `content` parameter used to load page fragments does not sanitize HTML markup before rendering, allowing arbitrary HTML to be injected into the page. This can be used to alter page appearance, insert fake forms for credential harvesting, or serve as a stepping stone toward XSS.

Steps to Reproduce

1.Go to: http://demo.testfire.net/index.jsp?content=<h1>HTML</h1>

2.Result: the injected `<h1>HTML</h1>` tag is rendered as a real, styled heading on the page rather than being displayed or stripped as plain text.

Screenshot / Proof of Concept (PoC)



Remediation

Sanitize/encode all output derived from the `content` parameter, and avoid using user-controllable input to select or render raw page fragments at all — use a fixed, server-side allow-list of valid page identifiers.

Finding #24: Server Status Check Service Exposed

Severity	Low
Affected URL	http://demo.testfire.net/util/serverStatusCheckService.jsp

Description & Impact

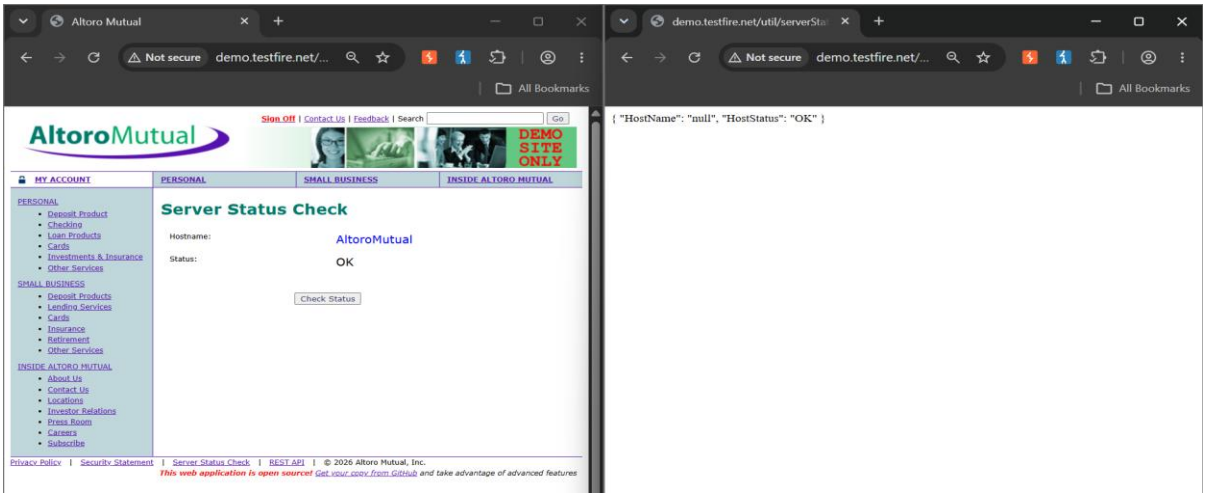
An internal-looking diagnostic/utility endpoint is reachable without authentication and returns server health/status information that should not be public.

Steps to Reproduce

1.Go to http://demo.testfire.net/util/serverStatusCheckService.jsp

2.Observe the server status output returned to an unauthenticated visitor.

Screenshot / Proof of Concept (PoC)



Remediation

Remove this endpoint from production or restrict it to internal monitoring systems / authenticated admins only.

Finding #25: Information Disclosure in HTML Source Comments

Severity	Low
Affected URL	http://demo.testfire.net/login.jsp

Description & Impact

The page's HTML source contains a developer comment revealing an internal contact process and a phone number for obtaining "the latest admin login," which is useful reconnaissance for social engineering / phishing against staff.

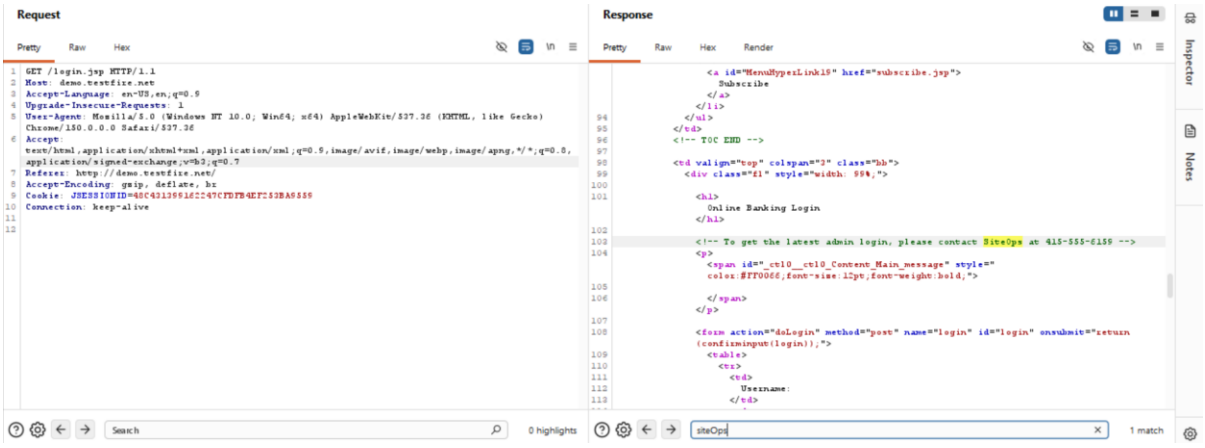
Steps to Reproduce

1.Go to the login page and view the page source (or intercept the response in a proxy).

2.Search the HTML for an HTML comment near the login header.

3.Result: an HTML comment is present referencing an internal contact process and phone number for admin login information.

Screenshot / Proof of Concept (PoC)



Remediation

Strip all internal notes, TODOs, and contact/process details from HTML shipped to production; keep such information in internal documentation only.

Finding #26: HTTP Strict Transport Security (HSTS) Header Missing

Severity	**Low**
Affected URL	http://demo.testfire.net/ (site-wide)

Description & Impact

The site does not send a `Strict-Transport-Security` header. Combined with Bug 21 (no HTTPS), this means browsers are never instructed to prefer/enforce a secure connection, leaving users open to protocol-downgrade and man-in-the-middle attacks.

Steps to Reproduce

1.Inspect the HTTP response headers for any page on the site.

2.Result: no Strict-Transport-Security header is present.

Remediation

Once the site is served over HTTPS, add the Strict-Transport-Security header with an appropriate max-age and includeSubDomains directive.

Finding #27: Referrer-Policy Header Missing

Severity	**Low**
Affected URL	http://demo.testfire.net/ (site-wide)

Description & Impact

No `Referrer-Policy` header is set, so the browser's default (often permissive) behavior is used, which can leak full URLs — potentially including sensitive query parameters — to third-party sites via the Referer header when users click outbound links.

Steps to Reproduce

1.Inspect the HTTP response headers for any page.

2.Result: no Referrer-Policy header is present.

Remediation

Add a Referrer-Policy header (e.g. strict-origin-when-cross-origin or no-referrer) to all responses.

Finding #28: Server Version Information Disclosure

Severity	Low
Affected URL	http://demo.testfire.net/ (Server response header)

Description & Impact

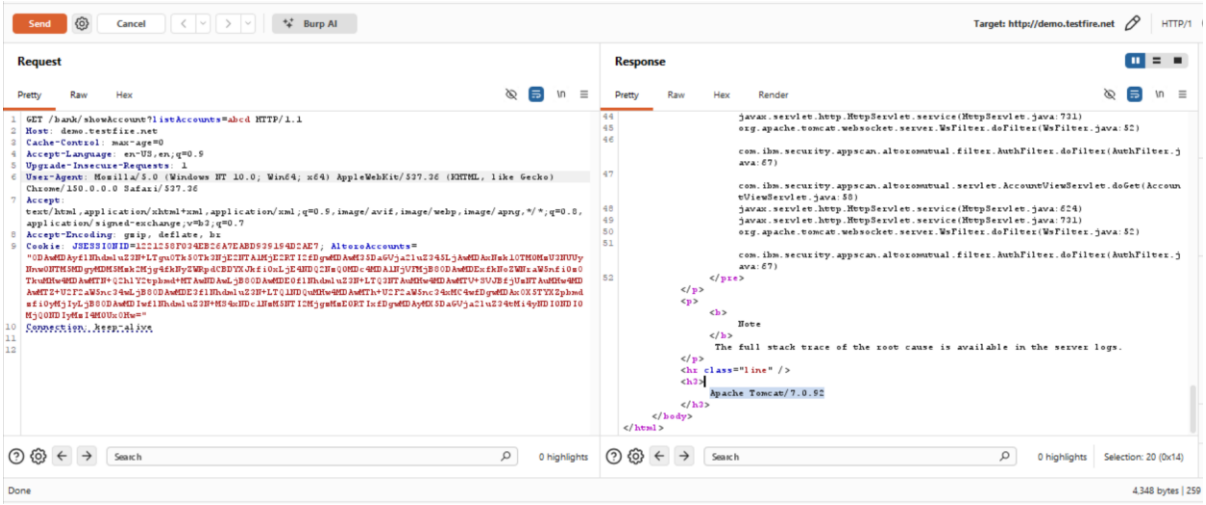
The HTTP `Server` response header discloses the exact application server and version in use — Apache Tomcat/7.0.92 — which is a known older version, making it easy for an attacker to look up applicable public exploits.

Steps to Reproduce

1.Inspect the HTTP response headers for any page.

2.Result: the Server header returns "Apache-Coyote/1.1" and the error pages additionally confirm "Apache Tomcat/7.0.92".

Screenshot / Proof of Concept (PoC)



Remediation

Suppress or generalize the Server header (and equivalent version banners) at the server/proxy level, and keep the underlying server software patched/current.

Finding #29: Information Disclosure — Internal Executives & Management Details

Severity	Low
Affected URL	http://demo.testfire.net/index.jsp?content=inside_executives.htm

Description & Impact

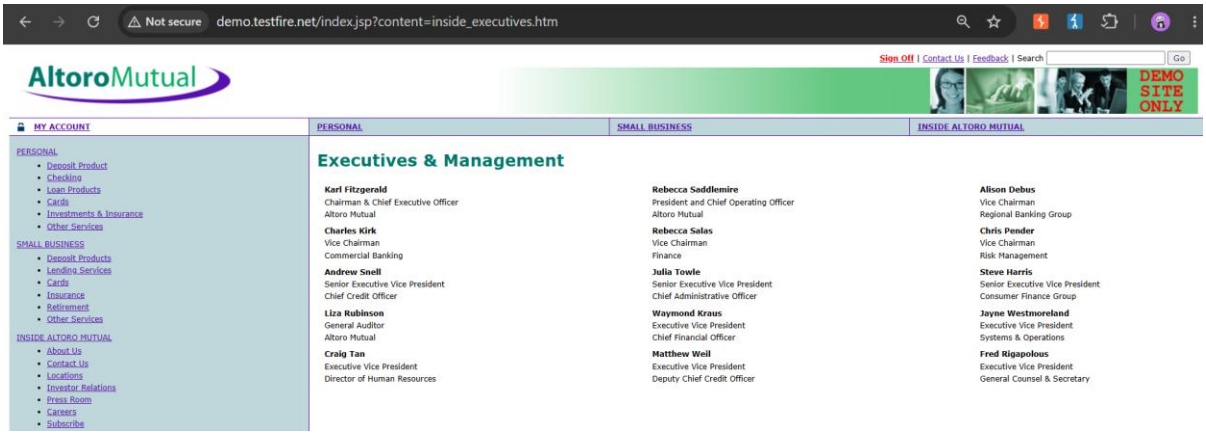
An internal-looking page listing company executives and management details is reachable by any visitor through the same `content` parameter mechanism, without authentication. Disclosing named individuals and their roles gives an attacker material for targeted phishing and social-engineering attacks.

Steps to Reproduce

1.Go to: http://demo.testfire.net/index.jsp?content=inside_executives.htm

2.Result: a page listing internal executive/management names and details loads without requiring login.

Screenshot / Proof of Concept (PoC)



Remediation

Require authentication for any internal-facing content, and audit all values reachable through the `content` parameter mechanism for similarly sensitive pages.

---

5. Overall Recommendations

- Fix all Critical findings first (see the top of the Detailed Findings section): #1 Admin Panel Accessible Without Authorization, #2 SQL Injection — Login Authentication Bypass, #3 IDOR — Unauthorized Access to Other Users' Bank Accounts, #4 Broken Access Control — Fund Transfer Request Tampering.
- Move the entire site to HTTPS with HSTS enabled, and stop storing sensitive account data in client-side cookies.
- Add output encoding everywhere user input is reflected (search, feedback form, content parameter) to close all XSS and HTML-injection findings at once.
- Add the missing security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy) and CSRF tokens on all state-changing forms.
- Remove or restrict all unauthenticated diagnostic/legacy endpoints (Swagger UI, server status check, cgi.exe reference, admin panel) from the public-facing environment.
- Turn off verbose error/stack-trace output in production and use generic error pages.
